SmartCity Pulse – Documentação de Arquitetura e Implementação (AWS)

Plataforma de monitoramento urbano e IoT em tempo real: ingestão de telemetria de sensores, detecção de anomalias, alertas automatizados, ocorrências de cidadãos e relatórios analíticos. Documento técnico completo para implementar o projeto de ponta a ponta como **projeto de estudo e portfólio em engenharia de nuvem**, com foco em arquitetura, segurança, observabilidade, CI/CD e **controle de custos**.

1. Visão geral e objetivos

Conceito: uma cidade inteligente simulada. Milhares de sensores (trânsito, poluição, câmeras) e cidadãos (chamados de ocorrência) geram eventos contínuos. A plataforma os recebe, processa em tempo real, dispara alertas em caso de anomalia e oferece relatórios e painéis.

Objetivos de aprendizado (mapa para as habilidades mais pedidas em nuvem):

Habilidade	Onde aparece no projeto
Redes e segurança (VPC, subnets, IAM, Cognito, KMS)	Fundação da conta, rede em camadas e papéis de menor privilégio
Containers e orquestração	Microsserviços em ECS com Fargate atrás de um ALB
Streaming e mensageria	Kinesis, SQS, SNS e EventBridge
Bancos de dados	RDS PostgreSQL (transacional), DynamoDB (telemetria), S3 (data lake)
Serverless	Lambda acionada por S3 e por agendamento
Analytics	Firehose, S3 em Parquet, Glue e Athena
Observabilidade	CloudWatch, alarmes, dashboards e rastreamento com OpenTelemetry
DevOps e IaC	Terraform (ou CDK), GitHub Actions, ECR e implantação no ECS
FinOps	Orçamentos, etiquetas e rotina de destruição do ambiente

2. Revisão da proposta inicial (ajustes recomendados)

A arquitetura proposta já é sólida. Os pontos abaixo a deixam mais **atual, segura e barata**, e já estão incorporados no restante do documento.

Ponto da proposta	Observação	Ajuste adotado
AWS X-Ray para *distributed tracing*	Desde 25/02/2026, os **SDKs e o daemon do X-Ray estão em modo de manutenção** (apenas correções críticas), com fim de suporte em 25/02/2027. A AWS recomenda **OpenTelemetry** (AWS Distro for OpenTelemetry, ADOT), enviando os dados ao X-Ray	Instrumentar com **OpenTelemetry + coletor ADOT em *sidecar*** no ECS
Ingestão de sensores direto no Kinesis	Sensores reais costumam falar MQTT; **AWS IoT Core** é o serviço nativo para isso	**Decisão registrada em ADR:** simulador publica direto no Kinesis (mais simples e barato); IoT Core como extensão (seção 19)
NAT Gateway nas subnets privadas	É uma das maiores fontes de custo surpresa em ambientes de estudo (cobrança por hora e por GB)	Um único NAT em dev, **VPC endpoints** gratuitos (S3 e DynamoDB) e destruição do ambiente fora das sessões
Kinesis, SQS e SNS sem papéis definidos	São complementares, não intercambiáveis	Tabela de decisão na seção 4.3
DynamoDB para telemetria sem modelagem	Chaves mal escolhidas causam partições quentes e consultas caras	Modelo na seção 5, com *write sharding*, TTL e histórico no S3
RDS e DynamoDB como únicas estruturas de dados	Falta camada analítica para os "relatórios gerenciais"	**Firehose → S3 (Parquet) → Athena** para análises sem sobrecarregar o transacional
Autenticação e segredos não citados	RDS citado para "usuários"	**Cognito** para identidade, **Secrets Manager** para credenciais e **KMS** para criptografia
Infraestrutura criada "manualmente"	Difícil de reproduzir e de destruir	**IaC com Terraform (ou CDK)** desde a Fase 0

Ponto da proposta	Observação	Ajuste adotado
Pipeline com CodePipeline ou GitHub Actions	Ambos servem	**GitHub Actions com OIDC** (sem chaves de acesso fixas) como padrão; CodePipeline como alternativa
"Rolling update" no ECS	Funciona, mas sem proteção automática	Habilitar **circuit breaker de implantação** e avaliar implantação *blue/green* nativa do ECS
Câmeras de monitoramento	Imagens de pessoas e veículos envolvem **LGPD**	Usar **imagens sintéticas ou públicas**, sem reconhecimento facial (seção 16)
Custos não discutidos	O plano gratuito atual de contas novas é baseado em créditos, e não em horas grátis	Seção 12 inteira dedicada a FinOps

3. Requisitos

3.1 Requisitos funcionais

ID	Requisito
RF-01	Receber **telemetria contínua** de sensores de trânsito, poluição e (simulados) de câmeras
RF-02	Armazenar a telemetria bruta com **baixa latência de escrita** e consulta por sensor, zona e período
RF-03	**Detectar anomalias** por regras (limiares, variação brusca, sensor silencioso) e por desvio estatístico
RF-04	**Disparar alertas** com severidade, deduplicação e ciclo de vida (aberto, reconhecido, resolvido)
RF-05	**Notificar** operadores por e-mail, SMS ou *webhook*
RF-06	Cidadãos registram **ocorrências** com descrição, localização e foto

ID	Requisito
RF-07	Processar **imagens** enviadas (validação, miniatura e, opcionalmente, rótulos automáticos)
RF-08	Painel operacional com situação por zona, mapa e alertas ativos
RF-09	**Relatórios gerenciais** periódicos e sob demanda (por exemplo, diário e semanal)
RF-10	Cadastro de sensores, zonas, regras de alerta e usuários com **perfis** (cidadão, operador, gestor, administrador)
RF-11	API pública documentada (OpenAPI) com autenticação

3.2 Requisitos não funcionais (metas de estudo)

ID	Categoria	Meta
RNF-01	Latência de ingestão	Evento gravado em até 5 s (p99) após a publicação
RNF-02	Latência de alerta	Alerta emitido em até 10 s após a anomalia
RNF-03	API	p95 abaixo de 300 ms para consultas típicas
RNF-04	Disponibilidade	99,5% mensal no ambiente de demonstração
RNF-05	Escalabilidade	Suportar 10 vezes o volume de teste base apenas com ajustes de configuração
RNF-06	Segurança	Privilégio mínimo, criptografia em trânsito e em repouso, sem segredos no código
RNF-07	Resiliência	Falha de uma tarefa ou de uma AZ não interrompe a ingestão; mensagens com falha vão para DLQ
RNF-08	Observabilidade	Logs estruturados, métricas, alarmes e rastreamento ponta a ponta
RNF-09	Custo	Teto mensal definido e alertas em 50%, 80% e 100% do orçamento
RNF-10	Reprodutibilidade	Ambiente criado e destruído com um comando

3.3 Volume de referência e dimensionamento

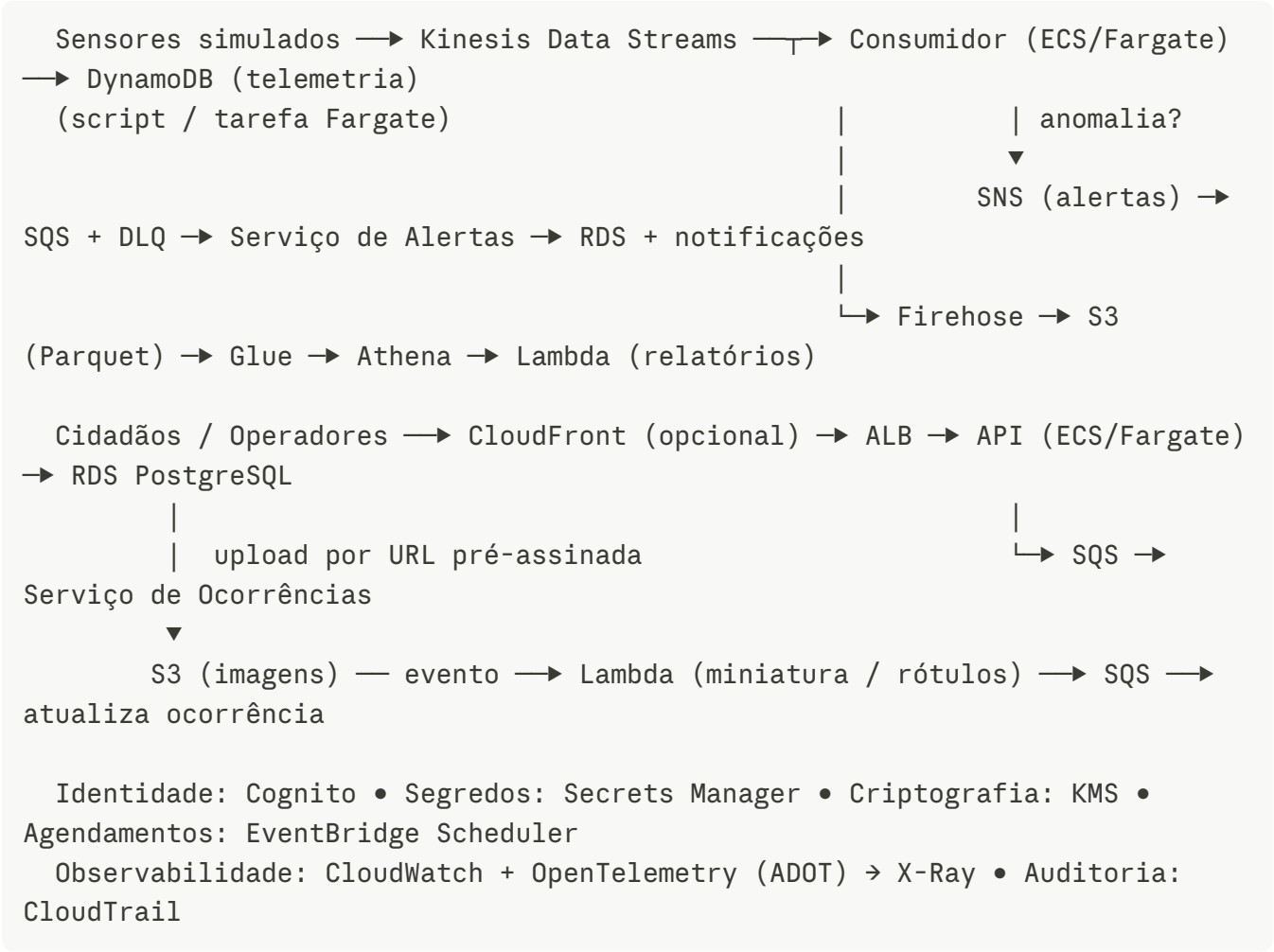
Exemplo para o ambiente de teste: **5.000 sensores**, um evento a cada 10 segundos, cerca de 0,5 KB por evento.

Cálculo	Resultado
Eventos por segundo	5.000 ÷ 10 = **500 eventos/s**
Vazão	500 × 0,5 KB = **≈ 250 KB/s**
Kinesis (cada *shard* aceita até 1 MB/s ou 1.000 registros/s de escrita)	**1 *shard*** é suficiente; 2 dão folga
Volume diário	500 × 86.400 ≈ 43 milhões de eventos ≈ **21 GB/dia** brutos

O volume diário já mostra por que a **retenção no DynamoDB deve ser curta (TTL)** e o histórico deve ir para o **S3 em formato colunar**. Para estudar, rode com **uma fração** (por exemplo, 100 a 500 sensores) e faça testes de carga curtos para o volume completo.

4. Arquitetura

4.1 Visão geral



4.2 Fluxos principais

Telemetria e alerta

1. O simulador publica lotes de eventos no **Kinesis** usando o `sensorId` como chave de partição (mantém a ordem por sensor).
2. O **consumidor** (ECS) lê o stream, valida o evento, grava no **DynamoDB** e executa a **detecção de anomalias**.
3. Se houver anomalia, publica no **SNS**, que entrega a uma **fila SQS** (com DLQ).
4. O **serviço de alertas** consome a fila, deduplica, grava no **RDS** e notifica (e-mail, SMS ou *webhook*).
5. Em paralelo, o **Firehose** entrega os mesmos eventos ao **S3** em Parquet para análises.

Ocorrência de cidadão com foto

1. O app chama `POST /ocorrencias`; a API grava no **RDS** e devolve uma **URL pré-assinada** de upload.
2. O cliente envia a foto direto ao **S3** (a API não trafega o arquivo).
3. O evento do S3 aciona uma **Lambda**, que valida o tipo, gera miniatura e (opcional) extrai rótulos.
4. A Lambda publica o resultado em uma **fila SQS**; o **serviço de ocorrências** atualiza o RDS. A Lambda **não precisa estar na VPC**, evitando NAT e conexões diretas ao banco.

Relatórios

1. O **EventBridge Scheduler** aciona uma Lambda todas as noites.
2. A Lambda executa consultas no **Athena** sobre o S3, grava o relatório (CSV ou PDF) no S3 e avisa por SNS com um link temporário.

4.3 Kinesis, SQS, SNS e EventBridge: quando usar cada um

Serviço	Papel	Uso no projeto	Características
Kinesis Data Streams	Fluxo ordenado de alto volume, com vários consumidores e reprocessamento	Telemetria dos sensores	Ordem por chave de partição; retenção configurável; cobrança por *shard* ou por uso
SQS	Fila de trabalho com tentativas e DLQ	Processar alertas, ocorrências e resultados de imagem	Desacopla produtor e consumidor; cada mensagem é processada por um consumidor
SNS	Publicação/assinatura (*fan-out*)	Distribuir alertas para fila, e-mail, SMS e *webhook*	Um evento, vários destinos
EventBridge	Barramento de eventos e agendador	Agendar relatórios e eventos de domínio	Regras, filtros e *Scheduler*

4.4 Serviços por camada

Camada	Serviços	Papéis e observações
Computação	**ECS + Fargate**, **Lambda**	Microsserviços contínuos e tarefas orientadas a eventos
Ingestão e mensageria	Kinesis, Firehose, SQS, SNS, EventBridge	Seção 4.3
Dados	RDS PostgreSQL, DynamoDB, S3, Glue, Athena	Transacional, telemetria, objetos e analytics
Rede e acesso	VPC, ALB, Route 53 e CloudFront (opcionais), WAF	Isolamento, entrada HTTPS e proteção
Identidade e segurança	IAM, Cognito, Secrets Manager, KMS, CloudTrail, GuardDuty (opcional)	Menor privilégio, usuários, segredos e auditoria
Observabilidade	CloudWatch, **OpenTelemetry (ADOT) + X-Ray**	Logs, métricas, alarmes e *traces*
DevOps	ECR, GitHub Actions (ou CodePipeline e CodeBuild), Terraform	Imagens, entrega contínua e infraestrutura como código

5. Modelo de dados

5.1 Evento de telemetria (contrato)

```
{
  "schemaVersion": 1,
  "eventId": "01J9ZK3Q7M",
  "sensorId": "poll-0042",
  "tipo": "poluicao",
  "zona": "centro-norte",
  "ocorridoEm": "2026-10-25T14:03:07Z",
  "medidas": { "pm25": 38.2, "pm10": 61.0, "no2": 22.4 },
  "bateria": 87
}
```

Use **versionamento do esquema** e **idempotência** por `eventId` (reentregas do Kinesis e do SQS são possíveis).

5.2 DynamoDB (telemetria e estado)

Tabela	Chave	Atributos e uso
`telemetria`	PK: `sensorId`; SK: `ocorridoEm` (ISO 8601)	`tipo`, `zona`, `medidas`, `expiraEm` (**TTL**, por exemplo, 7 dias). Consulta: "últimas leituras do sensor"
`telemetria` (GSI1)	PK: `zona#tipo#shard` (shard de 0 a 9); SK: `ocorridoEm`	Consulta "últimos minutos por zona". **O *write sharding* evita partição quente**, pois a escrita por zona seria concentrada
`estado_sensor`	PK: `sensorId`	Última leitura, último sinal de vida (*heartbeat*), estado do detector; base do alerta "sensor silencioso"
`idempotencia`	PK: `eventId`	Escrita condicional (`attribute_not_exists`) com TTL curto

Boas práticas: modo sob demanda no início, gravações em lote (`BatchWriteItem`), itens pequenos e **histórico de longo prazo no S3**, e não no DynamoDB.

5.3 RDS PostgreSQL (transacional)

Tabela	Campos principais
`zonas`	id, nome, geometria (ou centroide)
`sensores`	id, tipo, zona_id, status, instalado_em
`regras_alerta`	id, tipo_sensor, metrica, operador, limiar, janela_s, severidade, ativa
`alertas`	id, regra_id, sensor_id, zona_id, severidade, status, aberto_em, reconhecido_em, resolvido_em, **dedupe_key** (única por alerta aberto)
`notificacoes`	id, alerta_id, canal, destino, status, tentativas
`ocorrencias`	id, usuario_id, tipo, descricao, latitude, longitude, status, criada_em, foto_s3_key, miniatura_s3_key, rotulos
`usuarios`	id, cognito_sub, nome, perfil

Tabela	Campos principais
`relatorios`	id, tipo, periodo, s3_key, gerado_em

Boas práticas: migrações versionadas (Flyway ou Liquibase), pool de conexões, índices para as consultas do painel e, se necessário, extensão PostGIS para consultas geográficas.

5.4 Organização dos buckets S3

Bucket ou prefixo	Conteúdo	Ciclo de vida
`telemetria-raw/ano=/mes=/dia=/`	Eventos em Parquet (via Firehose), particionados por data	Transição para classes mais baratas e expiração conforme a política
`ocorrencias-imagens/`	Fotos originais e miniaturas	Retenção definida e acesso por URL pré-assinada
`relatorios/`	Relatórios gerados	Expiração em dias
`artefatos/`	Estado do Terraform, *builds*, *backups*	Versionamento ativado

Todos com **Block Public Access**, criptografia (SSE-KMS ou SSE-S3), versionamento quando fizer sentido e política exigindo TLS.

6. Rede e segurança

6.1 VPC em camadas

Camada	Subnets	Recursos
Pública (2 AZs)	Rotas para o Internet Gateway	ALB e NAT Gateway
Aplicação (privada, 2 AZs)	Saída por NAT e por *endpoints*	Tarefas Fargate (API, consumidor, alertas, ocorrências)
Dados (privada, 2 AZs)	**Sem rota para a internet**	RDS

Controle	Configuração
Security Groups	ALB (443 da internet) → API (somente do SG do ALB) → RDS (porta 5432 somente dos SGs dos serviços)
Endpoints de VPC	**Gateway** para S3 e DynamoDB (sem custo); *interface endpoints* (ECR, CloudWatch Logs, Kinesis, SQS, Secrets Manager etc.) só se justificarem o custo por hora frente ao NAT
NAT	Um NAT em dev (menor custo) e um por AZ em produção; alternativa de estudo: NAT em instância, com menor disponibilidade
HTTPS	Certificado no ACM, redirecionamento 80 → 443 e política TLS atual
WAF	Regras gerenciadas e limite de taxa no ALB ou CloudFront

6.2 IAM (privilégio mínimo)

Papel	Pode	Não pode
Execução das tarefas ECS	Baixar imagem do ECR, escrever logs, ler segredos específicos	Acessar dados de negócio
`task-api`	Ler e escrever no RDS (via segredo), gerar URLs pré-assinadas do bucket de imagens, publicar na fila de ocorrências	Ler o stream de telemetria
`task-consumidor`	`kinesis:GetRecords` no stream, escrever nas tabelas do DynamoDB, publicar no tópico de alertas	Acessar o RDS
`task-alertas`	Consumir a fila de alertas, acessar o RDS, publicar notificações	Escrever no DynamoDB de telemetria
`lambda-imagem`	Ler o bucket de imagens, escrever miniaturas, publicar na fila	Acessar o RDS
`lambda-relatorio`	Consultar o Athena, ler dados e escrever relatórios	Alterar dados de origem

Papel	Pode	Não pode
`github-deploy` (OIDC)	Enviar imagem ao ECR e atualizar o serviço ECS	Administrar a conta

Restrinja cada política ao **ARN** do recurso e use condições quando possível. **Sem chaves de acesso de longa duração:** use papéis, IAM Identity Center para pessoas e OIDC para o CI.

6.3 Identidade, segredos e criptografia

- **Cognito:** grupos `cidadao`, `operador`, `gestor` e `admin`; tokens JWT validados na API; verificação de e-mail e política de senha.
- **Secrets Manager:** credenciais do RDS com rotação; nenhuma senha em variável de ambiente do código ou no repositório.
- **KMS:** chaves gerenciadas pelo cliente para RDS, S3 e logs sensíveis, quando o custo se justificar.
- **Em trânsito:** TLS entre cliente e ALB e entre serviços e bancos.
- **Auditoria:** CloudTrail ativo, com os logs em bucket protegido.
- **Conta:** MFA no usuário raiz, sem uso rotineiro dele, e acesso por IAM Identity Center.

7. Microsserviços e funções

Componente	Tipo	Responsabilidade
`api-publica`	ECS/Fargate atrás do ALB	Autenticação, consultas do painel, cadastro de sensores e regras, criação de ocorrências e URLs pré-assinadas
`consumidor-telemetria`	ECS/Fargate (sem ALB)	Ler o Kinesis, validar, gravar no DynamoDB, detectar anomalias e publicar no SNS
`servico-alertas`	ECS/Fargate (sem ALB)	Consumir a fila, deduplicar, gerenciar o ciclo de vida do alerta e notificar
`servico-ocorrencias`	ECS/Fargate (pode ser parte da API)	Atualizar ocorrências com os resultados do processamento de imagem
`simulador`	Tarefa Fargate agendada ou script local	Gerar telemetria sintética com anomalias controladas

Componente	Tipo	Responsabilidade
`lambda-imagem`	Lambda (gatilho S3)	Validação, miniatura e rótulos opcionais
`lambda-relatorio`	Lambda (EventBridge Scheduler)	Consultas Athena e geração de relatórios

Stack sugerida: Node.js com TypeScript (Fastify ou NestJS) ou Java com Spring Boot, a seu critério; os exemplos usam TypeScript e Python. Cada serviço tem sua imagem Docker, *health checks* (`/health` e `/ready`), configuração por variáveis de ambiente, encerramento gracioso (*graceful shutdown*) e logs em JSON.

Escala automática: a API escala por CPU ou por requisições no ALB; os consumidores escalam pelo número de *shards* (uma tarefa por *shard* é o máximo útil) e a fila de alertas escala pela profundidade da fila.

8. Detecção de anomalias e alertas

8.1 Estratégias

Estratégia	Exemplo	Observação
Limiar fixo	PM2,5 acima de um limite por mais de N minutos	Regra configurável no RDS
Taxa de variação	Salto brusco entre leituras	Pega falhas e eventos súbitos
Desvio estatístico (EWMA e z-score)	Leitura muito fora do padrão recente do sensor	Adapta-se ao comportamento de cada sensor
Sensor silencioso	Nenhuma leitura em X minutos	Verificação periódica sobre `estado_sensor`
Trânsito	Velocidade média baixa com contagem de veículos alta na mesma zona	Regra composta por zona
Correlação por zona	Vários sensores da mesma zona em alerta	Reduz falsos positivos de um sensor defeituoso

```
export class DetectorEwma {
  private media = 0;
  private variancia = 0;
  private n = 0;
```

```
  constructor(
    private alfa = 0.1,
    private limiteZ = 4,
    private aquecimento = 30,
  ) {}

  avaliar(valor: number): { anomalia: boolean; z: number } {
    if (this.n === 0) {
      this.media = valor;
      this.n = 1;
      return { anomalia: false, z: 0 };
    }
    const desvio = Math.sqrt(this.variancia) || 1e-9;
    const z = (valor - this.media) / desvio;
    const anomalia = this.n >= this.aquecimento && Math.abs(z) >
this.limiteZ;

    // só aprende com valores normais, para não "normalizar" a anomalia
    if (!anomalia) {
      const delta = valor - this.media;
      this.media += this.alfa * delta;
      this.variancia = (1 - this.alfa) * (this.variancia + this.alfa * delta
* delta);
    }
    this.n++;
    return { anomalia, z };
  }
}
```

Como o `sensorId` é a chave de partição, todas as leituras de um sensor chegam ao mesmo consumidor, que pode manter o estado do detector em memória e **persistir um *checkpoint* periódico** em `estado_sensor` para sobreviver a reinicializações.

8.2 Ciclo de vida do alerta

- **Estados:** `ABERTO` → `RECONHECIDO` → `RESOLVIDO` (ou `AUTO_RESOLVIDO` quando a métrica normaliza).
- **Deduplicação:** `dedupe_key = regra + sensor + janela`; enquanto houver alerta aberto com a mesma chave, o app atualiza o contador em vez de criar outro.
- **Histerese:** limiares diferentes para abrir e para fechar, evitando alertas intermitentes.
- **Escalonamento:** alerta não reconhecido em X minutos sobe de severidade e notifica o nível seguinte.

- **Idempotência:** processar a mesma mensagem duas vezes não gera notificação duplicada.

9. Infraestrutura como código e ambientes

```
infra/
├── envs/
|   ├── dev/        # main.tf, variables.tf, backend.tf
|   └── prod/
└── modules/
    ├── network/    # VPC, subnets, rotas, endpoints, NAT
    ├── ecs-service/
    ├── kinesis/
    ├── dynamodb/
    ├── rds/
    ├── s3/
    ├── lambda/
    ├── cognito/
    ├── observability/
    └── iam/
```

Prática	Detalhe
Estado remoto	Estado do Terraform no S3 com versionamento e bloqueio
Módulos	Um módulo por responsabilidade, com variáveis e saídas claras
Ambientes	`dev` barato e descartável; `prod` (simulado) com mais redundância
Etiquetas (*tags*)	`projeto`, `ambiente`, `dono` e `centro-de-custo` em todos os recursos, para rastrear custos
Verificações	`terraform fmt`, `validate`, `plan` no *pull request* e análise de segurança (por exemplo, tfsec, Checkov ou Trivy)
Destruição	`terraform destroy` ao fim de cada sessão de estudo, preservando apenas o que for barato (estado e artefatos)
Conta	Idealmente contas separadas por ambiente; no mínimo, separação por prefixos, etiquetas e papéis

> **Alternativa:** AWS CDK (TypeScript) se quiser infraestrutura na mesma linguagem do backend. Escolha uma ferramenta e registre a decisão em um ADR.

10. Observabilidade

10.1 Logs, métricas e rastreamento

Pilar	Prática
Logs	JSON estruturado no CloudWatch Logs com `timestamp`, `nivel`, `servico`, `correlationId`, `sensorId` e `eventId`; **retenção curta** (por exemplo, 7 dias) para controlar custo; nunca registrar dados pessoais ou segredos
Métricas	Métricas padrão (ECS, ALB, RDS, Kinesis, SQS, DynamoDB) + métricas próprias via *Embedded Metric Format*: eventos processados, latência de ingestão, anomalias detectadas, mensagens na DLQ
Rastreamento	**OpenTelemetry** com o coletor **ADOT em *sidecar*** no ECS, enviando *traces* ao X-Ray. Propague o contexto (`traceparent`) nos atributos das mensagens SNS e SQS para ligar o rastro ponta a ponta
Correlação	O `correlationId` nasce na borda (ALB ou simulador) e aparece em logs, métricas e *traces*

> O X-Ray continua sendo o destino dos *traces*; o que mudou foi a **instrumentação**: os SDKs e o daemon clássicos estão em manutenção desde fevereiro de 2026, e a recomendação para projetos novos é OpenTelemetry.

10.2 Alarmes essenciais

Alarme	Métrica	Quando disparar (ponto de partida)
Atraso do consumidor	Kinesis `GetRecords.IteratorAgeMilliseconds`	Crescente por alguns minutos (o consumidor não acompanha o stream)
Escritas limitadas	Kinesis `WriteProvisionedThroughputExceeded`	Maior que zero de forma sustentada
DLQ com mensagens	SQS `ApproximateNumberOfMessagesVisible` na DLQ	Maior que zero
Fila envelhecendo	SQS `ApproximateAgeOfOldestMessage`	Acima do SLO de alerta
Erros 5xx	ALB `HTTPCode_Target_5XX_Count`	Taxa acima do limite
Latência	ALB `TargetResponseTime` p95	Acima de 300 ms
Tarefas saudáveis	ALB `HealthyHostCount` ou ECS `RunningTaskCount`	Abaixo do desejado
Banco	RDS `CPUUtilization`, `DatabaseConnections`, `FreeStorageSpace`	Valores próximos do limite
DynamoDB	`ThrottledRequests` e erros de sistema	Maior que zero
Orçamento	AWS Budgets	50%, 80% e 100% do teto

Os alarmes notificam um tópico SNS (e-mail ou *webhook*) e cada um aponta para um **runbook** curto: o que verificar e como mitigar.

10.3 Dashboards

- **Operacional:** vazão e atraso do stream, erros, latência, tarefas, fila e DLQ.
- **Negócio:** sensores ativos e silenciosos, alertas por zona e severidade, ocorrências abertas.
- **Custos:** gasto diário por serviço e por etiqueta.
- Opcional: **CloudWatch Synthetics** (*canary*) chamando a API a cada poucos minutos.

11. CI/CD

11.1 Estágios do pipeline

Estágio	Ações	Falha bloqueia?
Validação	Formatação, *lint*, tipos, testes unitários	Sim
Segurança	Análise de dependências, varredura da imagem (Trivy ou varredura do ECR), análise do Terraform (tfsec ou Checkov), detecção de segredos	Sim para severidade alta
Build	Imagem Docker com *tag* imutável (SHA do *commit*)	Sim
Publicação	Envio ao **ECR**	Sim
Implantação em dev	Atualização do serviço ECS com **circuit breaker** e *rollback* automático	Sim
Testes pós-implantação	*Smoke tests* contra o ALB e verificação de saúde	Sim
Produção (simulada)	Aprovação manual e implantação (rolling ou *blue/green*)	Sim

11.2 Exemplo com GitHub Actions e OIDC

```
name: deploy-api
on:
  push:
    branches: [main]

permissions:
  id-token: write   # necessário para o OIDC
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # use as versões estáveis vigentes
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<CONTA>:role/github-deploy
```

```
          aws-region: us-east-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build e push
        run: |
          IMAGEM=${{ steps.ecr.outputs.registry }}/smartcity-api:${{
github.sha }}
          docker build -t $IMAGEM .
          docker push $IMAGEM
      - name: Atualizar serviço
        run: |
          # registra nova definição de tarefa com a imagem e atualiza o
serviço
          aws ecs update-service --cluster smartcity-dev --service api --
force-new-deployment
```

11.3 Práticas

- **Sem chaves fixas:** o papel `github-deploy` é assumido por OIDC, com política de confiança restrita ao repositório e à *branch*.
- **Infraestrutura em pipeline separado:** `terraform plan` no *pull request* e `apply` após aprovação.
- **Migrações de banco** como etapa controlada (tarefa ECS única ou ferramenta no pipeline), nunca na inicialização de várias réplicas ao mesmo tempo.
- **Rollback:** voltar para a definição de tarefa anterior; com *blue/green*, desviar o tráfego de volta.
- **Promoção:** a mesma imagem (mesmo SHA) sobe de dev para produção.

12. Custos e FinOps

Em projetos de estudo na nuvem, **o custo é o maior risco real**. Trate-o como requisito.

12.1 Modelo atual de contas novas

Contas AWS criadas a partir de 15/07/2025 usam um modelo **baseado em créditos**, e não mais os 12 meses gratuitos de antes. Ao abrir a conta, escolhe-se entre:

Plano	Como funciona (resumo)
Gratuito	Créditos iniciais de US$ 100, podendo chegar a US$ 200 ao concluir atividades de onboarding. Dura até **6 meses ou o fim dos créditos**, o que vier primeiro; depois disso a conta é **encerrada** se não for atualizada para o plano pago. Alguns serviços mais caros ou voltados a grandes empresas ficam indisponíveis. Não gera cobrança além dos créditos enquanto estiver nesse plano
Pago	Acesso a todos os serviços, com os créditos aplicados automaticamente; após acabarem, a cobrança segue o modelo normal por uso

> Confirme o modelo vigente, os serviços disponíveis no plano gratuito e os prazos na documentação oficial e no console (*Billing*). Esta arquitetura (NAT, ALB, Fargate, Kinesis e RDS) **consome créditos rapidamente** se ficar ligada 24 horas; por isso a rotina de destruição é obrigatória. Contas anteriores a essa data continuam no modelo antigo.

12.2 Principais geradores de custo

Recurso	Por que custa	Como reduzir no estudo
NAT Gateway	Cobrança por hora e por GB processado	Um único NAT em dev, *endpoints* gratuitos para S3 e DynamoDB e **destruição fora das sessões**
ALB	Cobrança por hora e por uso	Um ALB compartilhado por várias rotas; destruir quando não usar
Fargate	vCPU e memória por segundo	Menores tamanhos de tarefa, arquitetura ARM (Graviton), **Fargate Spot** para o simulador e (em dev) o consumidor, e mínimo de réplicas
RDS	Hora de instância, armazenamento e *backups*	Menor instância, **sem Multi-AZ em dev**, parada quando ociosa (atenção: o RDS parado volta a ligar sozinho após alguns dias)

Recurso	Por que custa	Como reduzir no estudo
Kinesis	Hora de *shard* (provisionado) ou uso (sob demanda)	**1 *shard* provisionado** em dev, e apagar o stream ao fim
DynamoDB	Por requisição (sob demanda)	Simular uma fração dos sensores e fazer o teste de carga completo apenas por minutos
CloudWatch Logs	Por GB ingerido e armazenado	Menos verbosidade, retenção curta e amostragem
Endpoints de interface	Por hora e por AZ	Usar só quando justificarem frente ao NAT
Athena	Por dado escaneado	Parquet, partições por data e colunas selecionadas
Recursos "esquecidos"	Volumes, IPs elásticos, *snapshots* e logs	Revisão após o `destroy` e etiquetas para localizá-los

12.3 Salvaguardas

- **Estimar antes de criar:** usar a AWS Pricing Calculator para cada fase e anotar a estimativa no ADR.
- **AWS Budgets** com alertas em 50%, 80% e 100% e, se possível, ações automáticas; **Cost Anomaly Detection** ativo.
- **Etiquetas obrigatórias** (`projeto` , `ambiente` , `dono`) e relatórios no Cost Explorer por etiqueta.
- **Script de "desligar tudo"**: escalar os serviços a zero, parar o RDS, apagar o NAT e o stream, e conferir recursos remanescentes.
- **Rotina de sessão:** subir com Terraform, estudar, validar e **destruir**; registrar o custo real de cada sessão e compará-lo com a estimativa.
- Nunca guardar chaves de acesso de longa duração: um vazamento pode gerar cobranças altas.

13. Testes e qualidade

Tipo	Escopo	Ferramentas
Unitários	Detector de anomalias, regras de alerta, validação de eventos, deduplicação	Jest ou Vitest (TypeScript); JUnit (Java)

Tipo	Escopo	Ferramentas
Contrato	Esquema JSON do evento (versão 1 e compatibilidade) e contrato da API (OpenAPI)	JSON Schema, Pact ou testes de esquema
Integração	Consumidores com fila, banco e DynamoDB locais (com as limitações de fidelidade de emuladores)	Testcontainers, LocalStack
E2E em dev	Simulador → stream → alerta → notificação, e ocorrência com foto → miniatura	Scripts e *smoke tests*
Carga	Produtor com 500 eventos/s por 10 a 15 minutos; API com k6	k6, Artillery ou o próprio simulador
Resiliência	Matar uma tarefa, mensagem inválida (*poison message*), limitação do DynamoDB, indisponibilidade do banco	Experimentos manuais documentados
Segurança	Imagens, dependências, IaC e permissões (IAM Access Analyzer)	Trivy, tfsec/Checkov, Access Analyzer

Metas do teste de carga: atraso do consumidor (*iterator age*) estabilizado, nenhuma mensagem perdida (contagem enviada = contagem gravada), latência dentro do RNF-01 e custo da execução registrado.

14. Simulador de sensores

Aspecto	Definição
Tipos	Poluição (PM2,5, PM10, NO2), trânsito (velocidade média e contagem) e câmeras (evento de imagem com referência no S3)
Perfil temporal	Picos nos horários de rush e variação ao longo do dia (senoide com ruído)
Anomalias	Injeção controlada por semente: pico de poluição, queda brusca de velocidade, sensor que para de enviar, leitura inválida
Escala	Parâmetros: número de sensores, intervalo e duração

Aspecto	Definição
Reprodutibilidade	Semente fixa para repetir o mesmo cenário nos testes
Execução	Script local ou tarefa Fargate agendada

```
import json, random, time
import boto3

kinesis = boto3.client("kinesis")
STREAM = "smartcity-telemetria"

def evento_poluicao(sensor_id: str, zona: str, anomalia: bool = False) ->
dict:
    pm25 = random.gauss(18, 4) + (random.uniform(60, 120) if anomalia else 0)
    return {
        "schemaVersion": 1,
        "eventId": f"{sensor_id}-{int(time.time() * 1000)}",
        "sensorId": sensor_id,
        "tipo": "poluicao",
        "zona": zona,
        "ocorridoEm": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "medidas": {"pm25": round(pm25, 1)},
    }

def publicar(eventos: list[dict]) -> None:
    for i in range(0, len(eventos), 500):          # limite de 500 registros
por chamada
        lote = eventos[i:i + 500]
        resp = kinesis.put_records(
            StreamName=STREAM,
            Records=[{"Data": json.dumps(e).encode(), "PartitionKey":
e["sensorId"]} for e in lote],
        )
        if resp["FailedRecordCount"]:
            # reenviar apenas os registros que falharam (com espera
crescente)
            ...
```

15. Roteiro de implementação por fases

O roteiro expande as cinco fases da proposta original em fases menores, com critério de aceite verificável.

Fase	Entrega	Critério de aceite	Duração (meio período)
0 – Fundação da conta e IaC	Conta segura (MFA, Identity Center), Budgets, repositório, estado remoto do Terraform, etiquetas, ADRs iniciais	`terraform plan` funciona; alertas de orçamento testados	3 a 5 dias
1 – Rede e dados	VPC em 3 camadas, *endpoints*, RDS PostgreSQL, tabelas do DynamoDB, buckets S3	Conexão ao RDS apenas de dentro da VPC; sem acesso público aos dados	1 semana
2 – API em container	API com `/health`, imagem no ECR, ECS Fargate, ALB com HTTPS, Cognito	Chamada autenticada via ALB responde; tarefa substituída sem queda	1 a 2 semanas
3 – Ingestão e mensageria	Stream, consumidor, detecção, SNS + SQS + DLQ, serviço de alertas, simulador	Anomalia injetada gera alerta único em até 10 s; reprocesso não duplica	2 semanas
4 – Serverless e storage	Upload por URL pré-assinada, Lambda de imagem, fila, atualização de ocorrências	Foto enviada vira miniatura e rótulo na ocorrência, sem Lambda na VPC	1 semana
5 – Analytics e relatórios	Firehose → S3 (Parquet), Glue, Athena, relatório agendado	Relatório diário gerado e enviado; consulta usa partições	1 semana
6 – Observabilidade e segurança	Logs, métricas, alarmes, dashboards, OpenTelemetry com ADOT, WAF, revisão de IAM	*Trace* completo de um alerta; todos os alarmes essenciais testados	1 a 2 semanas
7 – CI/CD	GitHub Actions com OIDC, publicação no ECR, implantação com *rollback*, pipeline do Terraform	`git push` leva à implantação sem intervenção; *rollback* demonstrado	1 semana

Fase	Entrega	Critério de aceite	Duração (meio período)
8 – Hardening e entrega	Teste de carga, testes de resiliência, relatório de custos, *Well-Architected review*, documentação e demo	Metas dos RNFs verificadas e documentadas	1 a 2 semanas

Total aproximado: 10 a 13 semanas em meio período (cerca de 150 a 220 horas). Para um MVP didático, as fases 0 a 3 já entregam o fluxo principal (telemetria até alerta).

16. Segurança, privacidade e boas práticas

16.1 Privacidade e LGPD

- **Dados pessoais:** nome, contato, localização e fotos de ocorrências são dados pessoais; colete o mínimo, informe a finalidade e permita exclusão.
- **Câmeras:** use **imagens sintéticas ou de domínio público**. Em casos reais, imagens de pessoas e placas exigem base legal, minimização e mascaramento. **Não implemente reconhecimento facial.** A detecção de rótulos genéricos (por exemplo, "veículo") é o máximo recomendado para estudo.
- **Retenção:** TTL no DynamoDB, ciclo de vida no S3 e expurgo de ocorrências antigas.
- **Logs:** sem dados pessoais; mascarar identificadores quando necessário.

16.2 Revisão pelos pilares do Well-Architected

Pilar	Práticas no projeto
Excelência operacional	IaC, CI/CD, runbooks, logs e métricas, ADRs
Segurança	Privilégio mínimo, VPC em camadas, criptografia, Cognito, WAF, CloudTrail
Confiabilidade	Multi-AZ em produção simulada, DLQ, reintentos idempotentes, *circuit breaker* de implantação, backups
Eficiência de desempenho	Kinesis com partição por sensor, *write sharding*, Parquet, escala automática

Pilar	Práticas no projeto
Otimização de custos	Orçamentos, etiquetas, destruição de ambientes, Fargate Spot e dimensionamento enxuto
Sustentabilidade	Recursos sob demanda, ARM (Graviton) e retenção curta de dados

17. Riscos e armadilhas

Risco	Prob.	Impacto	Mitigação
Custo inesperado (NAT, ALB, RDS, Kinesis ligados)	Alta	Alto	Budgets, estimativa prévia, `destroy` ao fim da sessão e script de desligamento
Conta no plano gratuito encerrada ao fim do prazo ou dos créditos	Média	Alto	Acompanhar créditos e decidir pelo plano pago com Budgets; manter IaC para recriar
Partição quente no DynamoDB ou no GSI	Média	Médio	*Write sharding* e modelagem por padrões de acesso
Consumidor atrasado (*iterator age* crescente)	Média	Alto	Alarme, escala por *shard* e processamento em lote
Mensagem inválida travando a fila	Média	Médio	DLQ, validação de esquema e *redrive* controlado
Reentrega duplicada de eventos	Alta	Médio	Idempotência por `eventId` e `dedupe_key`
Permissões IAM amplas demais	Média	Alto	Políticas por ARN, IAM Access Analyzer e revisão na Fase 6
Vazamento de credenciais	Baixa	Crítico	OIDC, Identity Center, Secrets Manager e detecção de segredos no CI
Ferramentas ou serviços que mudam (por exemplo, instrumentação do X-Ray)	Média	Médio	Acompanhar anúncios da AWS; registrar decisões em ADR
Excesso de serviços para o objetivo de aprendizado	Alta	Médio	Entregar por fases e priorizar o fluxo principal

Risco	Prob.	Impacto	Mitigação
Simulação pouco realista	Média	Baixo	Perfis temporais e anomalias parametrizadas
Dados pessoais em imagens reais	Baixa	Alto	Imagens sintéticas e sem reconhecimento facial

18. Entregáveis e critérios de sucesso

- **Repositório** (monorepo ou multi-repo) com `apps/`, `infra/`, `docs/` e `scripts/`.
- **Diagrama de arquitetura** (por exemplo, Mermaid ou diagrams.net) e diagrama de sequência dos fluxos principais.
- **ADRs:** (001) Fargate vs. EKS vs. EC2; (002) Kinesis vs. SQS vs. MSK; (003) modelagem do DynamoDB; (004) Terraform vs. CDK; (005) OpenTelemetry vs. SDK do X-Ray; (006) NAT vs. *endpoints*; (007) simulador direto no Kinesis vs. IoT Core; (008) GitHub Actions vs. CodePipeline; (009) Cognito; (010) retenção e LGPD.
- **Runbooks** dos alarmes e procedimento de recuperação.
- **Relatório de teste de carga** e de resiliência.
- **Relatório de custos** (estimado vs. real por fase) e plano de destruição.
- **README** com "como subir", "como testar" e "como destruir".
- **Vídeo de demonstração** (3 a 5 minutos): simulador gerando anomalia, alerta chegando, *trace*, dashboard e relatório.

Critérios de sucesso (metas de estudo, não garantias): ambiente criado e destruído por IaC; fluxo telemetria → alerta dentro da meta de latência; nenhuma mensagem perdida no teste de carga; alarmes essenciais testados; permissões revisadas; custo real dentro do orçamento definido.

19. Extensões futuras

- **AWS IoT Core** (MQTT, regras e *device shadow*) como porta de entrada realista dos sensores.
- **Amazon Managed Service for Apache Flink** para janelas e agregações em tempo real sobre o stream.
- **Detecção de anomalias com ML** (por exemplo, Random Cut Forest no SageMaker) comparada ao detector estatístico.
- **Step Functions** para fluxos de resposta a incidentes (acionar equipe, escalonar, encerrar).

- **Painel em tempo real** com WebSocket ou SSE; **Amazon Location Service** ou mapas open source para o mapa.
- **Amazon Managed Grafana** ou QuickSight para visualização.
- **Multi-região e recuperação de desastres** (replicação do S3, DynamoDB Global Tables e réplica do RDS).
- **Comparação com EKS** e com uma versão 100% serverless (API Gateway + Lambda), com análise de custo e operação.

20. Referências

- **AWS Well-Architected Framework** e seus pilares.
- Documentação oficial: Amazon ECS e Fargate, Kinesis Data Streams e Firehose, SQS, SNS, EventBridge, DynamoDB (melhores práticas de modelagem), RDS, S3, Lambda, VPC, IAM, Cognito, CloudWatch, Athena e Glue.
- Guia da AWS de **migração da instrumentação do X-Ray para OpenTelemetry** e documentação do **AWS Distro for OpenTelemetry**.
- Páginas oficiais do **AWS Free Tier**, **AWS Budgets** e **AWS Pricing Calculator**.
- Documentação do **Terraform** (provedor AWS) e do **GitHub Actions** para autenticação OIDC na AWS.
- DeBrie, A. — *The DynamoDB Book*; palestras de modelagem de DynamoDB do re:Invent.
- Kleppmann, M. — *Designing Data-Intensive Applications*.
- Newman, S. — *Building Microservices*.
- **LGPD** (Lei 13.709/2018) e orientações da ANPD.

21. Próximos passos

1. **Proteger a conta** antes de criar qualquer recurso: MFA no usuário raiz, IAM Identity Center, **AWS Budgets** com alertas e escolha consciente do plano (gratuito ou pago) e da região.
2. **Criar o repositório**, o estado remoto do Terraform e o padrão de etiquetas.
3. **Escrever os ADRs 001 a 003** e fazer a estimativa de custo da Fase 1 na Pricing Calculator.
4. Executar a **Fase 1** (rede e dados) com IaC e, ao fim da sessão, **destruir** o ambiente e anotar o custo real.
5. Seguir as fases 2 e 3 para ter o fluxo principal (telemetria até alerta) funcionando antes de expandir.